

第4章 软件架构的风格与模式

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第4章 软件架构的风格与模式

- 4.0 引言
- 4.1 软件架构风格定义
- 4.2 软件架构风格的分类
- 4.3 典型的软件架构风格
- 4.4 软件架构模式

哥特式风格





4.0 引言

- 哥特式建筑风格

- 强调垂直高度和石骨架结构
- 尖塔高耸
- 束柱
- 飞扶壁
- 尖顶形状的圆弧拱门
- 宗教意义的窗饰花纹和彩色玻璃

- 中国宫殿建筑风格

- 木构架体系，一般分为下、中、上三部分。
- 下部台基平整，基座华丽
- 中部由柱、梁、枋、檩、椽和斗拱构成的层叠式结构
- 顶分单檐、重檐，上面多盖琉璃瓦
- 多为建筑群

4.0 引言

Architectural style constitutes a mode of classifying architecture largely by morphological characteristics in terms of form, techniques, materials, etc.

建筑风格等同于建筑体系结构的一种可分类的模式，通过诸如外形、技术和材料等形态上的特征加以区分。

- “风格”——经过长时间的实践，被证明具有良好的工艺可行性、性能与实用性，并可直接用来遵循与模仿 (复用)。

4.1 软件架构风格的定义

- A software architecture style (软件体系结构风格)
 - describes a class of architectures (描述一类体系结构)
 - independent on the problems (独立于实际问题，强调了软件系统中通用的组织结构)
 - found repeatedly in practice (在实践中被多次应用)
 - a package of design decisions (是若干设计思想的综合)
 - has known properties that permit reuse (具有已经被熟知的特性，并且可以复用)

4.1 软件架构风格的定义

- 软件架构风格 (software architecture style) , 又称软件架构惯用模式 (software architecture idiomatic paradigm)
- 是描述**某一特定应用领域**中系统组织方式的**惯用模式**, 作为“**可复用的组织模式和习语**”, 为设计人员的交流提供了公共的术语空间, 促进了**设计复用与代码复用**。

4.1 软件架构风格的定义

- 架构风格的基本属性
 - 设计元素的词汇表（包括组件、连接件的类型以及数据元素，例如：管道、过滤器、对象、服务等）
 - 配置规则：决定元素组合的拓扑约束
 - 例如：限制某一风格中的组件至多与其它两个组件相连。
 - 元素组合的语义解释以及使用某种风格构建的系统的分析

4.1 软件架构风格的定义

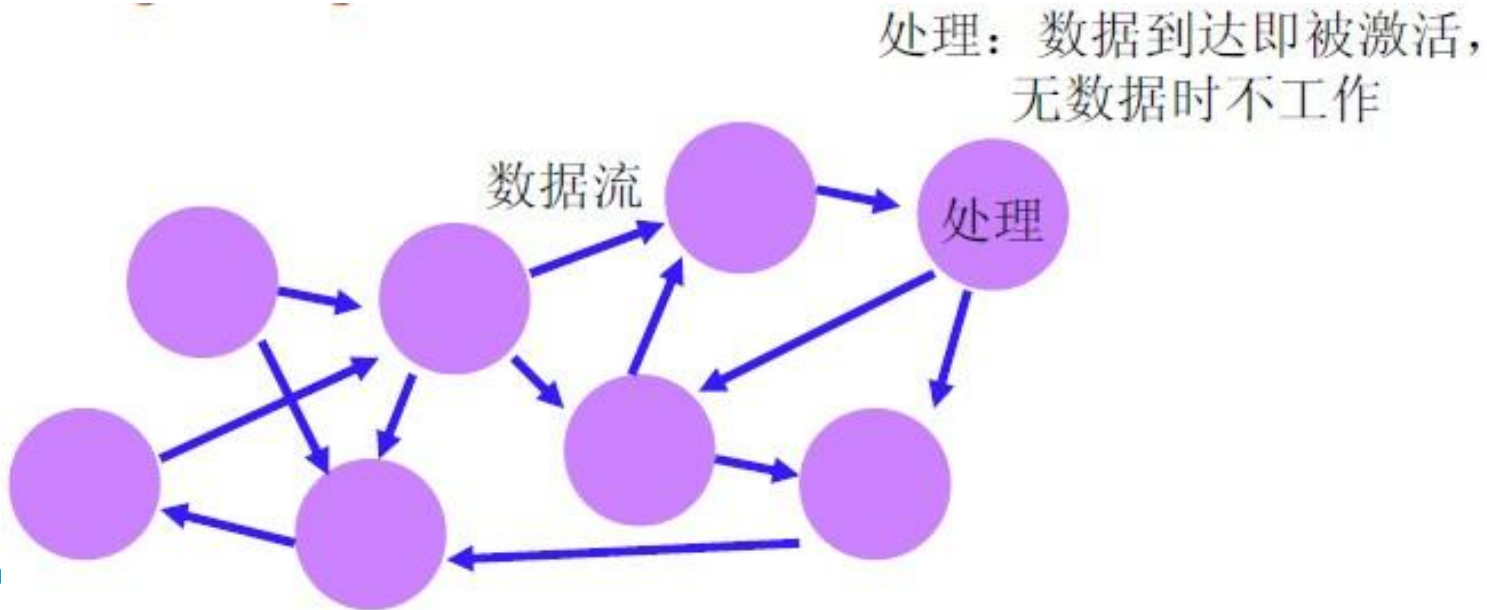
- 使用架构风格的优势
 - 可以极大地促进设计的重用性和代码的**重用性**，并且使得系统的组织结构**易被理解**。
 - 即使没有给出具体实现细节，依然可以通过“客户/服务器”架构风格大致推测出系统的组成结构和工作方式。
 - 使用标准的架构风格可较好地支持系统内部的**互操作性**以及**针对特定风格的分析**
 - 如：“管道/过滤器”风格可用来分析调度、吞吐量、延迟，和死锁等问题。

4.2 软件架构风格的分类

- (1) 数据流风格：批处理、管道/过滤器；
- (2) 调用/返回风格：主程序/子程序、面向对象风格、层次结构；
- (3) 以数据为中心的风格：仓库风格、超文本系统、黑板风格等；
- (4) 虚拟机风格：解释器、基于规则的系统；
- (5) 独立组件风格：进程通讯、事件系统。

4.3 典型的软件架构风格

• (1) 数据流风格

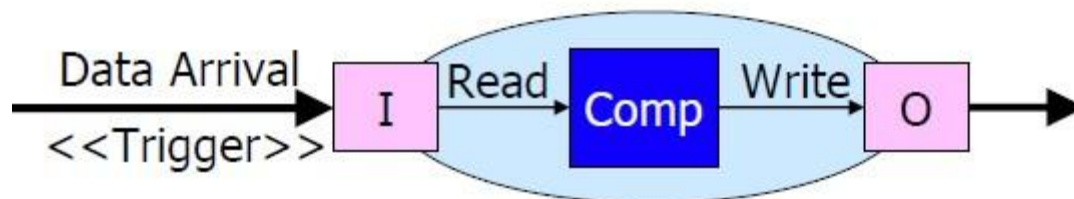


◦ 特征

- 数据的可用性决定着处理是否执行
- 系统结构由数据在各处理之间的有序移动决定
- 在纯数据流系统中，处理之间除了数据交换没有任何其他的交互

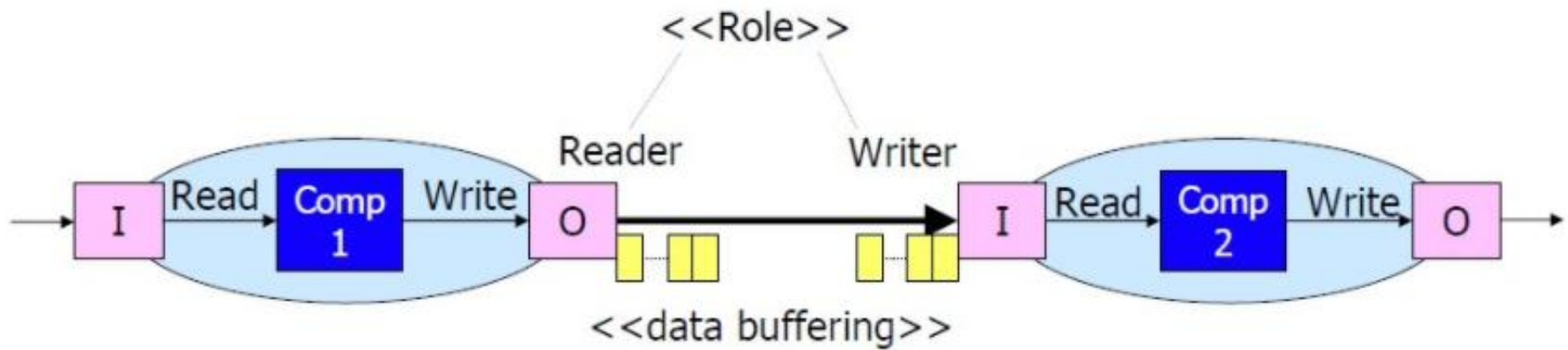
4.3 典型的软件架构风格

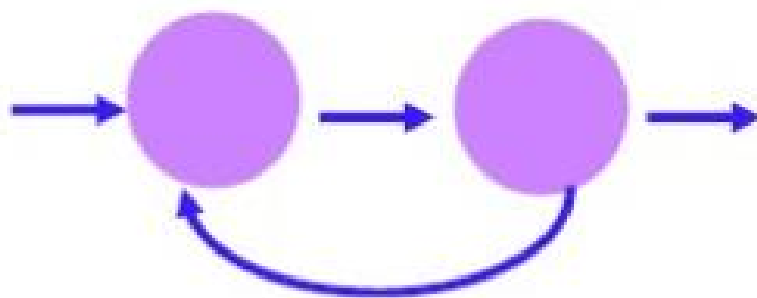
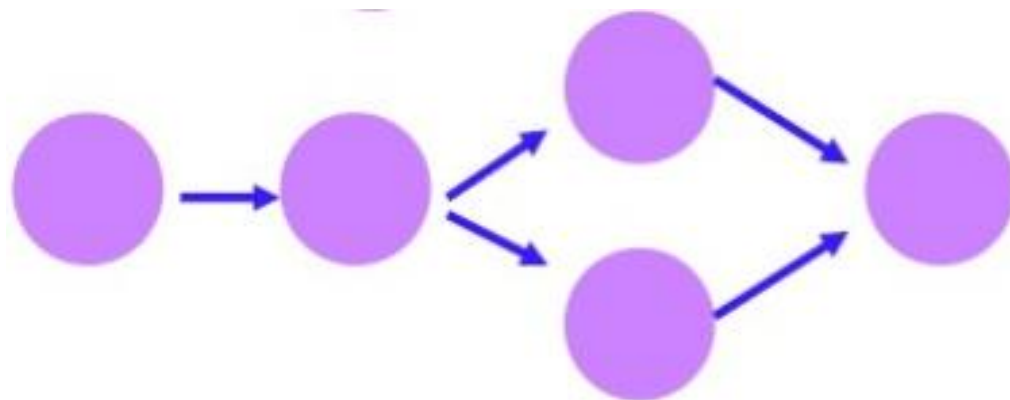
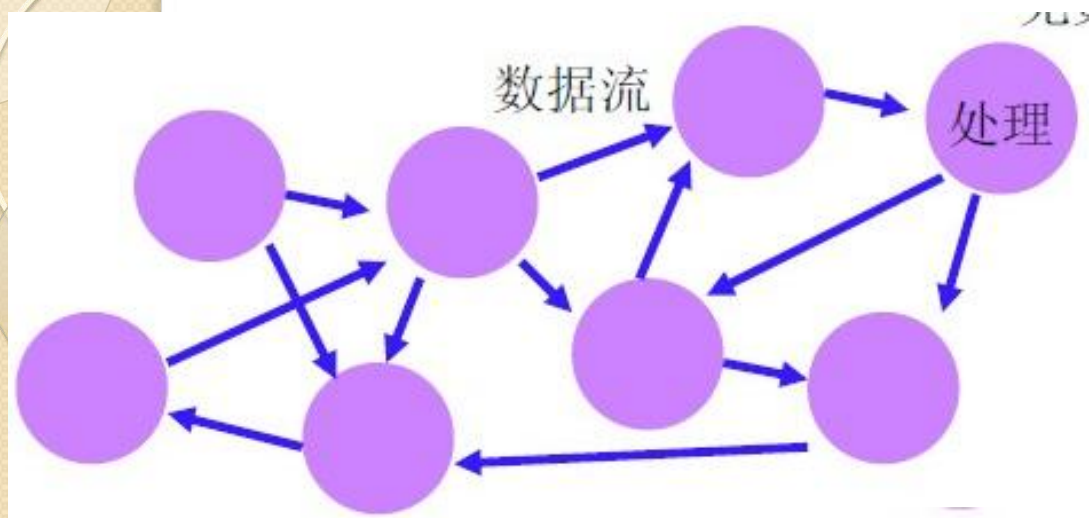
- (1) 数据流风格
 - 基本组件：数据处理单元 (Comp)
 - 组件的端口：输入端口 (I) 和输出端口 (O)
 - 组件的计算模型：从输入端口读数，经过计算/处理，然后写到输出端口



4.3 典型的软件架构风格

- (1) 数据流风格
 - 连接件：数据流
 - 接口角色：Reader和 Writer



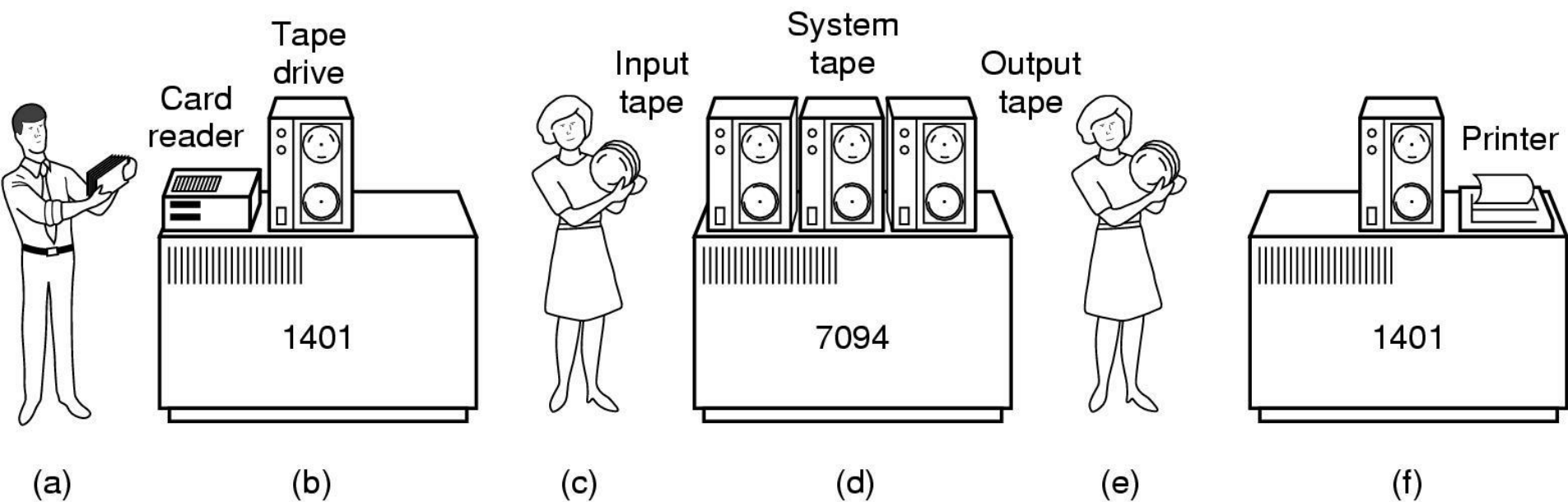


4.3 典型的软件架构风格

- (1) 数据流风格
- 两种典型的数据流风格：
 - 管道过滤器 (Pipe and Filter)
 - 批处理 (Batch Sequential)

4.3 典型的软件架构风格

- (I) 数据流风格
- 批处理 (Batch Sequential)



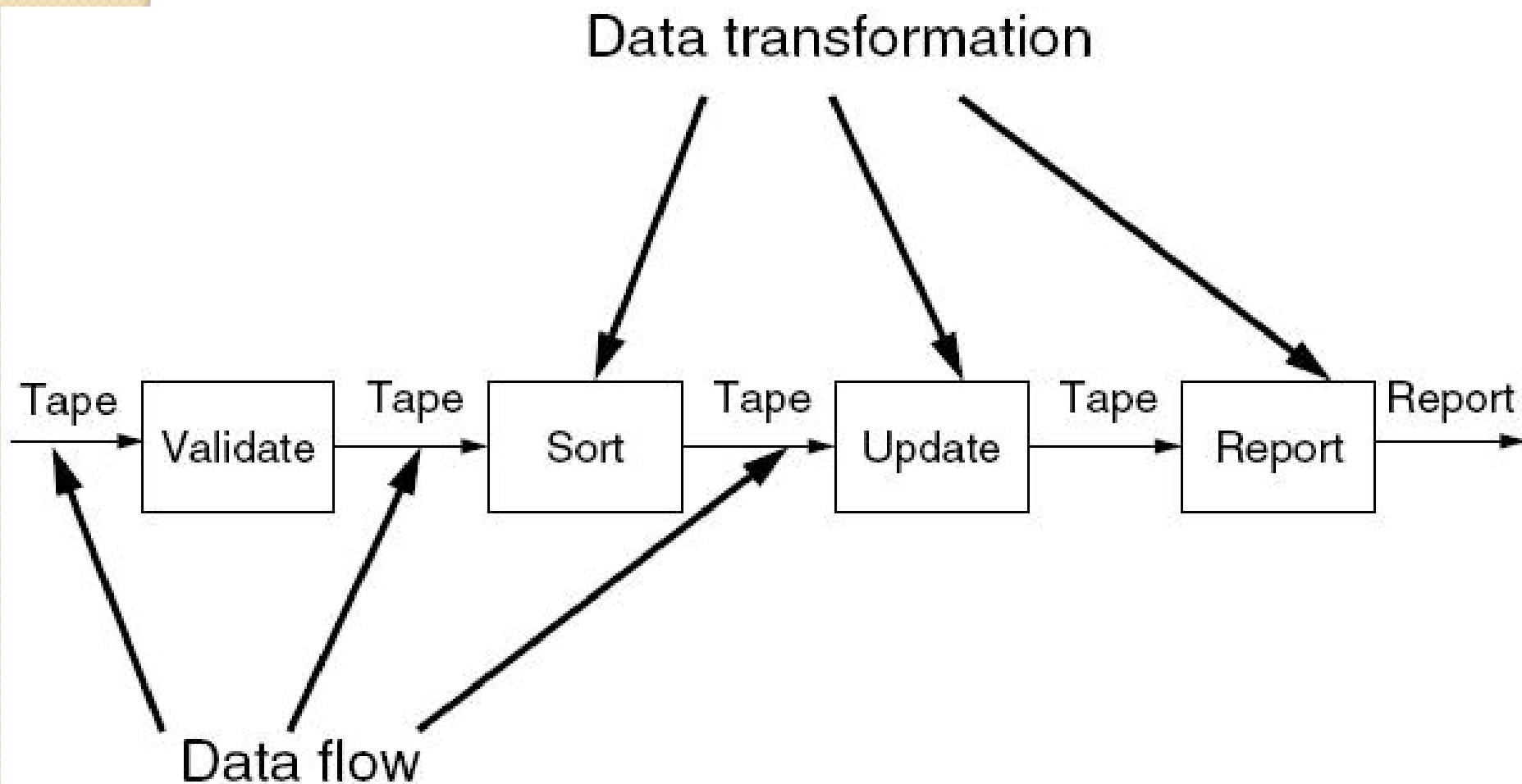
将用户数据写入磁带

磁带数据作为输入，计算后结果写入另一个磁带

输出结果打印

4.3 典型的软件架构风格

- (1) 数据流风格
- 批处理 (Batch Sequential)

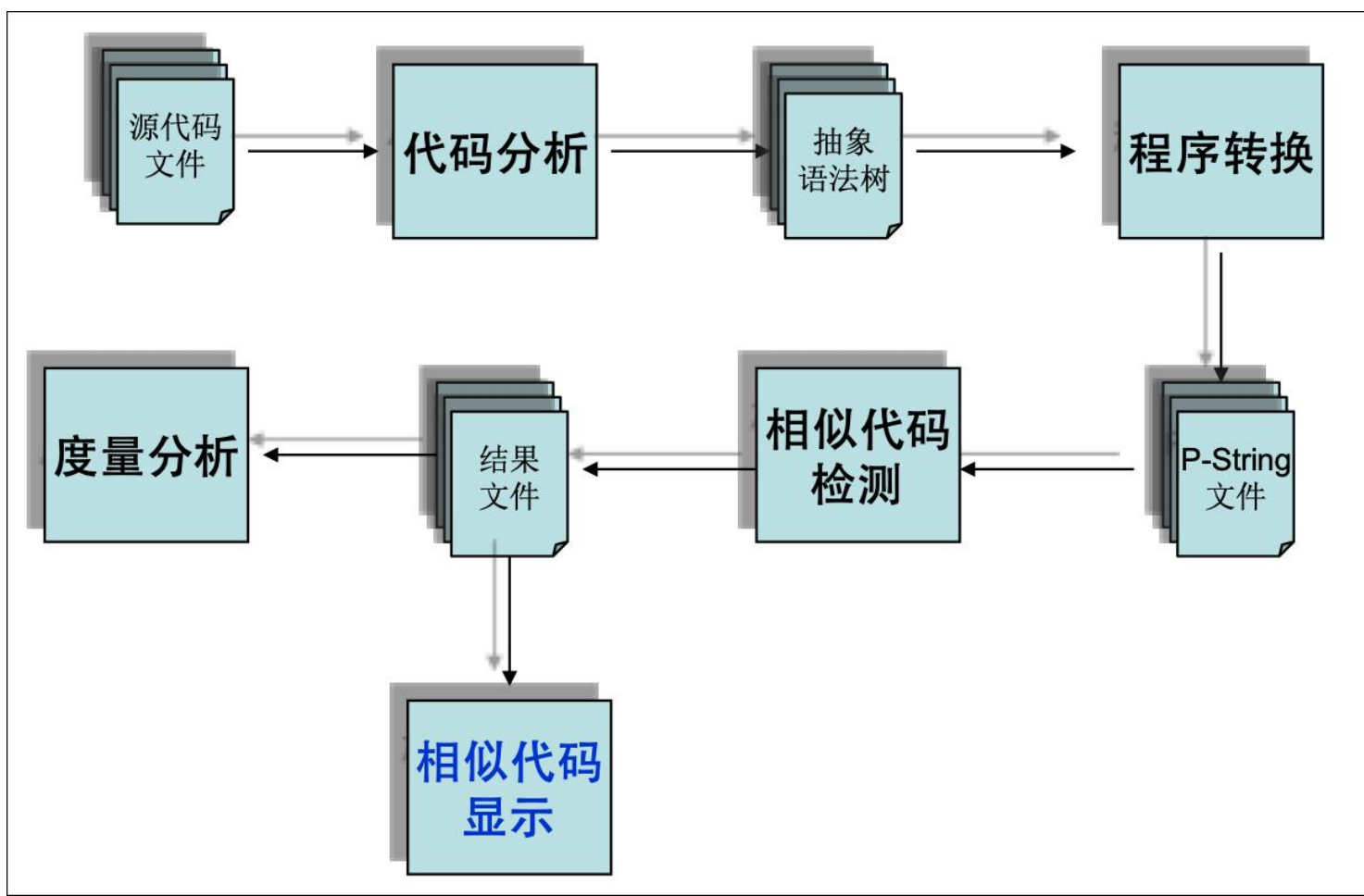


4.3 典型的软件架构风格

- (1) 数据流风格
- 批处理 (Batch Sequential)
 - 基本组件：独立的应用程序
 - 连接件：某种类型的介质，完整的数据
 - 特点：
 - 近乎线性；
 - 每个处理步骤是一个独立的程序；
 - 每一步在前一步结束后才开始；
 - 数据必须是完整的，以整体的方式传播。

4.3 典型的软件架构风格

- (1) 数据流风格：批处理
 - 实例：基于Eclipse的代码重复检测工具



4.3 典型的软件架构风格

- (I) 数据流风格：管道-过滤器 (Pipe-Filter)



4.3 典型的软件架构风格

- (1) 数据流风格：管道-过滤器 (Pipe-Filter)

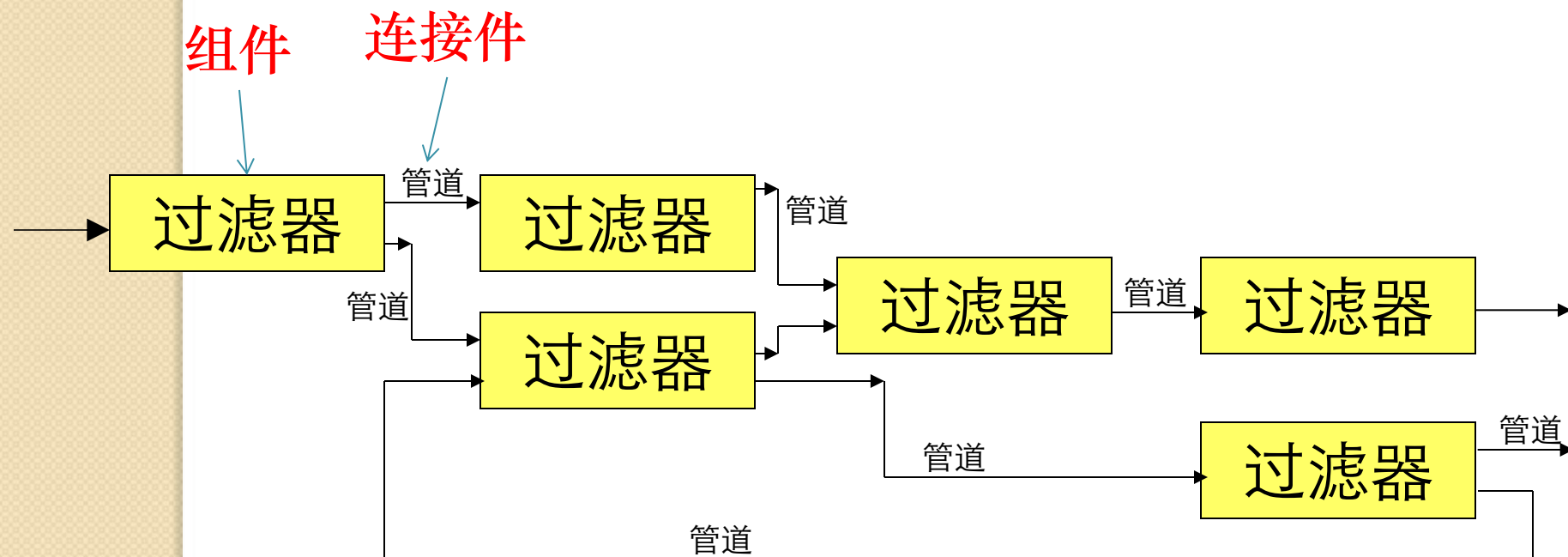


图4.1 管道 - 过滤器风格的体系结构

4.3 典型的软件架构风格

- (1) 数据流风格：管道-过滤器 (Pipe-Filter)
 - 应用场景：数据源源不断地产生，系统需要对这些数据进行若干处理(分析、计算、转换等)。
 - 基本组件：过滤器 (功能模块)
 - 每个过滤器组件中都封装了一个处理步骤
 - 数据源点和数据终止点可以看作是特殊的过滤器
 - 连接件：管道 (数据流)

4.3 典型的软件架构风格

- (1) 数据流风格
- 管道-过滤器 (Pipe-Filter)
 - 过滤器Filter:
 - 过滤器的作用：将源数据变换成目标数据
 - 过滤器的变换方式：（丰富）增加，（精练）删减，（转换）改变，分解，合并等
 - 过滤器的特性：独立性
 - 过滤器独立完成自身功能，相互之间无需进行状态交互。
 - 过滤器自身无状态
 - 过滤器对其上下游的过滤器“无知”

4.3 典型的软件架构风格

- (1) 数据流风格
- 管道-过滤器 (Pipe-Filter)
 - 管道Pipe:
 - 管道的作用：将数据从一个过滤器的输出口转移到另一个过滤器的输入口
 - 管道是单向流动的。
 - 管道可以有缓冲区。
 - 结果的正确性不依赖于各个过滤器运行的先后次序
 - 各过滤器在输入具备后完成自己的计算。完整的计算过程包含在过滤器之间的拓扑结构中。

管道-过滤器风格优点

- (1) 由于每个组件行为不受其他组件的影响，整个系统的行为易于理解。
 - 使得软件组件具有良好的**隐蔽性**和**高内聚、低耦合**的特点，允许设计者将整个系统的输入/输出行为看成是多个过滤器行为的简单合成；

管道-过滤器风格优点

- (2) 管道-过滤器风格支持**功能模块的复用**
 - 任何两个过滤器，只要它们之间传送的数据遵守共同的规约，就可以相连接。每个过滤器都有自己独立的输入输出接口，如果过滤器间传输的数据遵守其规约，只要用管道将它们连接就可以正常工作。

管道-过滤器风格优点

- (3) 基于管道-过滤器风格的系统具有较强的**可维护性**和**可扩展性**。
 - 旧的过滤器可以被替代，新的过滤器可以添加到已有的系统上。软件的易于维护和升级是衡量软件系统质量的重要指标之一，在管道-过滤器模型中，只要**遵守输入输出数据规约**，任何一个过滤器都可以被另一个新的过滤器代替，同时为增强程序功能，可以添加新的过滤器。这样，系统的可维护性和可升级性得到了保证。

管道-过滤器风格优点

- (4) 支持一些特定的分析，如**吞吐量计算**和**死锁检测**等。
 - 利用管道-过滤器风格的视图，可以很容易得到系统的资源使用和请求的状态图。然后，根据操作系统原理等相关理论中的死锁检测方法就可以分析出系统目前所处的状态，是否存在死锁可能及如何消除死锁等问题。

管道-过滤器风格优点

- (5) 管道-过滤器风格**具有并发性**
 - 每个过滤器作为一个单独的执行任务，可以与其它过滤器并发执行。过滤器的执行是独立的，不依赖于其它过滤器的。在实际运行时，可以将存在并发可能的多个过滤器看作多个并发的任务并行执行，从而大大提高系统的整体效率，加快处理速度。

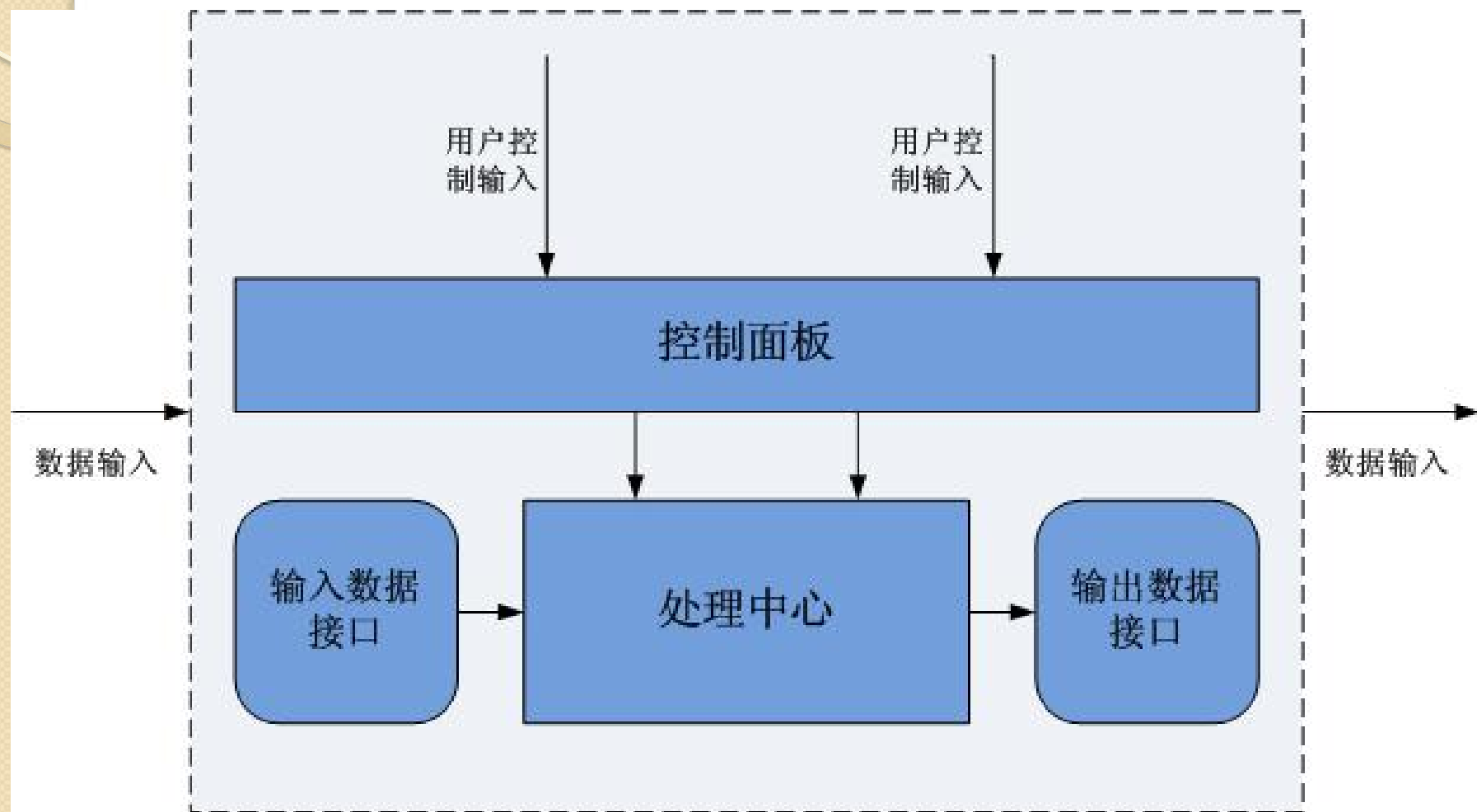
管道-过滤器风格不足

- (1) 管道-过滤器风格往往导致系统处理过程的成批操作。
- (2) 根据实际设计的需要，设计者也需要对数据传输进行特定的处理（如为了防止数据泄漏而采取加密等手段，或者使用了底层公共命名），导致过滤器必须对输入、输出管道中的数据流进行解析或反解析，增加了过滤器具体实现的复杂性，系统性能不高。

管道-过滤器风格不足

- (3) 交互式处理能力弱
 - 管道-过滤器模型适于数据流的处理和变换，不适合为与用户交互频繁的系统建模。
 - 系统没有一个共享的数据区，当涉及到多个过滤器需对同一个数据进行操作时，其实现较为复杂。

管道-过滤器风格不足



改进的过滤器

管道-过滤器风格实例

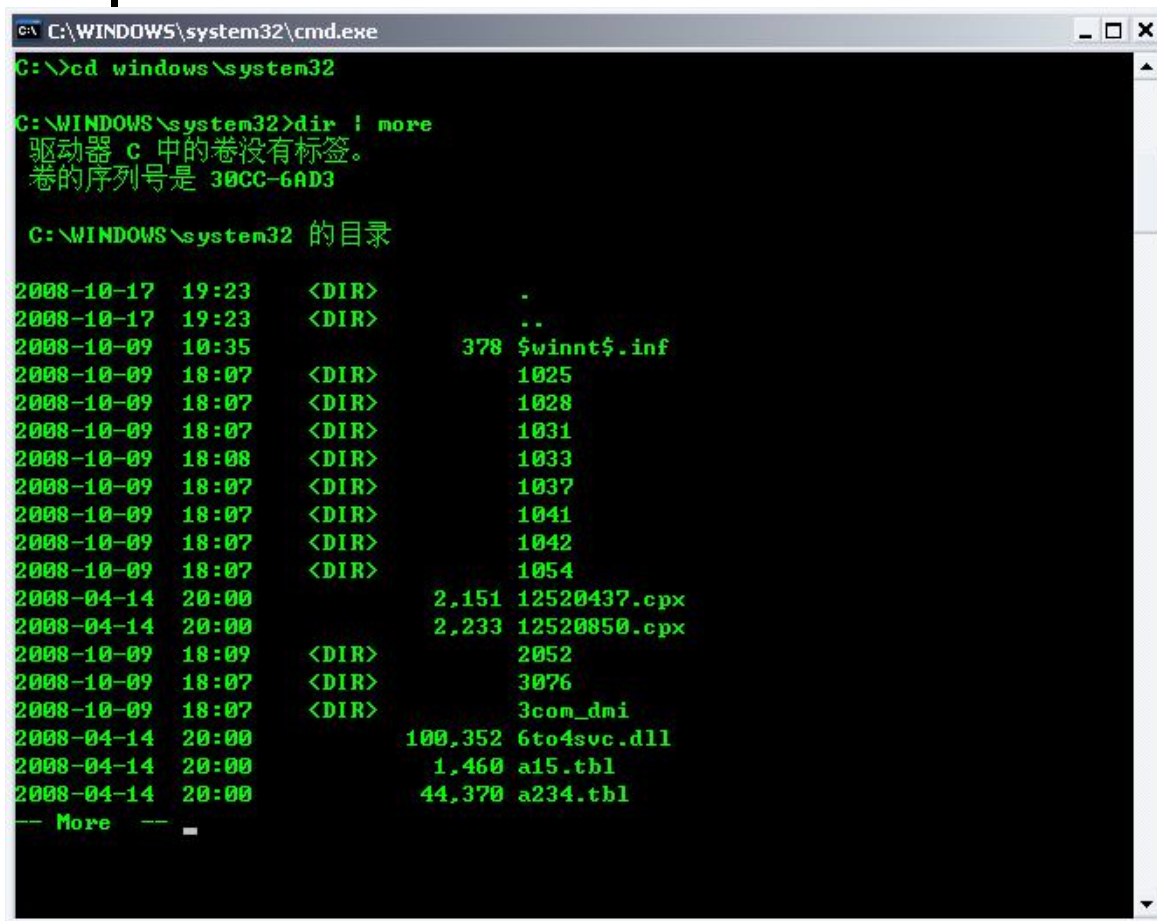
- 在Unix和Dos中，允许在命令中出现用竖线字符“|”分开的多个命令，将符号“|”之前的命令的输出，作为“|”之后命令的输入，这就是“管道功能”，竖线字符“|”是管道操作符。
- 例如： Unix shell中：
 - `cat file1|grep xyz|sort|uniq>out`
 - 即连接文件file1，找到含xyz的行，排序、去掉相同的行，最后输出

管道-过滤器风格实例

- 例如：DOS 中：
 - 命令 `dir | more` 使得当前目录列表在屏幕上逐屏显示。
 - `dir` 的输出是整个目录列表，它不出现在屏幕上而是由于符号 “|” 的规定，成为下一个命令 `more` 的输入，`more` 命令则将其输入，`more` 命令则将其输入一屏一屏地显示，成为命令行的输出。

管道-过滤器风格 实例

- dir | more



```
C:\WINDOWS\system32\cmd.exe
C:\>cd windows\system32

C:\WINDOWS\system32>dir | more
驱动器 c 中的卷没有标签。
卷的序列号是 30CC-6AD3

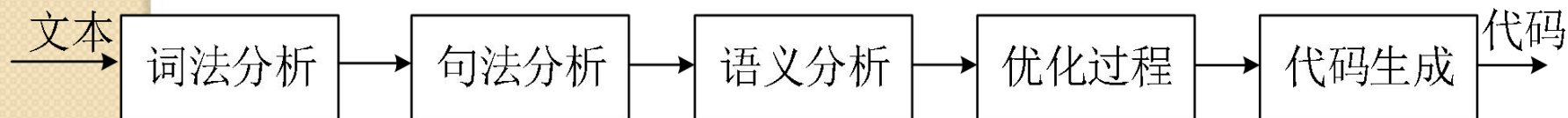
C:\WINDOWS\system32 的目录

2008-10-17  19:23    <DIR>          .
2008-10-17  19:23    <DIR>          ..
2008-10-09  10:35             378 $winnt$.inf
2008-10-09  18:07    <DIR>          1025
2008-10-09  18:07    <DIR>          1028
2008-10-09  18:07    <DIR>          1031
2008-10-09  18:08    <DIR>          1033
2008-10-09  18:07    <DIR>          1037
2008-10-09  18:07    <DIR>          1041
2008-10-09  18:07    <DIR>          1042
2008-10-09  18:07    <DIR>          1054
2008-04-14  20:00             2,151 12520437.cpx
2008-04-14  20:00             2,233 12520850.cpx
2008-10-09  18:09    <DIR>          2052
2008-10-09  18:07    <DIR>          3076
2008-10-09  18:07    <DIR>          3com_dmi
2008-04-14  20:00            100,352 6to4svc.dll
2008-04-14  20:00             1,460 a15.tbl
2008-04-14  20:00            44,370 a234.tbl
-- More --
```

管道-过滤器风格实例

- 实例2:

- 传统的编译器，一个阶段的输出是另一个阶段的输入（包括词法分析、语法分析、语义分析和代码生成）



4.3 典型的软件架构风格

- 数据流风格

批处理	管道过滤器
total(整体传递数据)	incremental(增量)
coarse grained(构件粒度较大)	fine grained (构件粒度较小)
high latency (延迟高, 实时性差)	results starts processing (实时性好)
no concurrency (无并发)	concurrency possible (可并发)

数据流风格应用

- Compiler (编译器)
- Unix pipes (Unix管道)
- Image processing (图像处理)
- Signal processing (信号处理)
- Voice and video streaming (声音与图像处理)
- ...
- 能够把任务分解成为一系列固定顺序的计算单元，并且彼此间只通过数据传递交互的场景均可应用

练习

- 假若要开发这样的软件：汽车牌照识别系统。
- 该系统一般可以分为：车辆图像获取、车辆牌照子图像定位、字符识别。

作业

- 调研一个后方交换型的 ATM 系统的结构，画出其体系结构图。

预习

- (1) 层次结构风格的特征是什么?
- (2) CS风格的结构特征、优缺点?
- (3) BS风格的结构特征、优缺点?
- (4) CS/BS风格的实例调研。
 - 它的需求背景，它为何要采用CS/BS风格?
 - 它的总体体系结构图
 - 说明它的使用是否能满足需求?
 - 你认为这样的设计决策是否合理?



*Thanks for
listening*